README

Mar 24, 2026

This project uses `uv` for Python environment management. It is a program that *does not depend on the local Python*. This means `uv` is the starting point of setting up a project, and it can install Python for the user, and manage both Python versions and packages.

1. Requirements

- The `uv` package manager should be installed following the install guide.
- The python version should already be specified in `pyproject.toml`, for example

```
requires-python = "~=3.11.0"
```

- Before running `uv sync`, there should be no `.venv` folder in the project directory. You can check this by running `ls -la`, and if you find an existing `.venv`, remove it with `rm -rf .venv`.
- The machine should be running MacOS, Linux x86-64, or Windows x86-64. Note that Linux arm-64 is not supported by `symbolica`.

2. Minimal Working Example

```
# Navigate to the project directory and install dependencies. The
# installation should start by installing python 3.11.x and then
proceed
# to install symbolica.

cd multipole_expansion
uv sync

# Once installation completes, you can run the demonstration scripts to
# see the multipole expansion in action.

uv run run_demo.py
uv run test_higher_orders.py

# OPTIONAL: if on Linux or MacOS, run
./run_demo.sh # to save the terminal output to run_demo.txt
```

The first script demonstrates the basic multipole expansion up to $n = 3$rd order and verifies that the Taylor expansion and Q tensor formulations agree. In its output, there are also comments and references to relevant python scripts. The second script pushes the implementation to higher orders, computing multipole moments up to $n = 7$.

3. Multipole Expansion

We briefly review multipole expansion before diving into the code.

Notation 3.1. The Einstein summation convention is used, except for the Taylor expansion. Tensors formed by juxtaposing vector components x^i, x^j, x^k, etc. are sometimes written as the vector's name with many indices like x^{ijk}, as is the standard shorthand.

Definition 3.2 (Green's function for static charge). The Green's function $\varphi_a(\vec{x}) := e_a/|\vec{x} - \vec{x}_a|$ is the solution to the point-charge Poisson equation

$$\nabla^2 \varphi_a = \delta^{(3)}(\vec{x} - \vec{x}_a). \tag{3.1}$$

Motivation 3.3. If $|\vec{x}_a| \ll |\vec{x}|$, physically we are observing at $\vec{x}$ which is "far away from the source", and hence can expand $\varphi(\vec{x})$ with respect to $\vec{x}_a$:

$$\varphi_a(\vec{x}) = \sum_{k=0}^{\infty} \frac{1}{k!} x_a^{i_1} ... x_a^{i_k} \frac{\partial^k}{\partial x_a^{i_1} \partial ... \partial x_a^{i_k}} \frac{e_a}{|\vec{x} - \vec{x}_a|} \quad \text{at } \vec{x}_a = 0 \tag{3.2}$$

Using substitution of variables

$$\frac{\partial}{\partial x_a^i} = (-1) \times \frac{\partial}{\partial (x^i - x_a^i)} \tag{3.3}$$

we can write the derivative part in Equation (3.2) as

$$\partial_{x_a^{i_q}} ... \partial_{x_a^{i_k}} \frac{1}{|\vec{x} - \vec{x}_a|} \bigg|_{\vec{x}=0} = (-1)^k \partial_{y^{i_1}} ... \partial_{y^{i_k}} \frac{1}{|\vec{y}|} \bigg|_{\vec{y}=\vec{x}}. \tag{3.4}$$

3.1. Derivative Tensors

Definition 3.4 (Derivative tensor). We define the derivative tensor of order k as

$$D_{i_1 \cdots i_k}(\vec{x}) := (-1)^k \partial_{y^{i_1}} ... \partial_{y^{i_k}} \frac{1}{|\vec{y}|} \quad \text{at } \vec{y} = \vec{x}. \tag{3.5}$$

Because partial derivatives commute, whenever a pair of indices i_p, i_q are contracted (i.e. traced over), we can commute them to the rightmost positions and use

$$\delta^{p,q} \partial_{i_p} \partial_{i_q} =: \nabla^2. \tag{3.6}$$

Result 3.5. The derivative tensor for the Green's function $\varphi_a(\vec{x})$ is traceless:

$$\forall (p, q), \operatorname{tr}_{(p,q)} D := \delta^{i_p, i_q} D = 0. \tag{3.7}$$

Remark 3.6. This traceless property comes from $\nabla^2(1/r) = 0$. Each source moment $x_a^{i_1} ... x_a^{i_k}$ has coefficient $D_{i_1 \cdots i_k}$ means: although we are expanding with respect to $\vec{x}_a$, the expansion is carried out around something rotation-invariant: the point charge at the origin.

Motivation 3.7. Let us now combine Definition 3.4 and Definition 3.2 to see how the expansion of φ can be expressed conveniently with D. For this purpose, we need to estimate the order of D in $|\vec{x}|$.

Inside D, the term with the lowest power of $|\vec{x}|$ is the one where every partial derivative operator acted on the $1/|\vec{x}|^{\cdots}$ term. From direct calculation, this term is

$$
\begin{aligned}
&(-1)^k\left(-\frac{1}{2}-0\right)\left(-\frac{1}{2}-1\right)\ldots\left(-\frac{1}{2}-(k-1)\right)\times 2^k\times y^{i_1}\ldots y^{i_k}\times\left(r^2\right)^{-1/2-k}\\
&=(2k-1)!!\times\frac{y^{i_1}\ldots y^{i_k}}{r^{2k+1}}\quad\text{with } r:=|\vec{x}|,\vec{y}=\vec{x}.
\end{aligned}
\tag{3.8}
$$

Starting from Equation (3.2), we insert D and manually multiply it with r^{2k+1} such that its explicit r dependence is in non-negative powers. We have

$$
\begin{aligned}
\varphi_a(\vec{x}) &= e_a\sum_{k=0}^{\infty}\frac{1}{k!}x_a^{i_1\ldots i_k}D_{i_1\ldots i_k}\\
&= e_a\sum_{k=0}^{\infty}\frac{1}{k!}\frac{x_a^{i_1\ldots i_k}}{r^k}\frac{D_{i_1\ldots i_k}\times r^{2k+1}}{r^{k+1}},
\end{aligned}
\tag{3.9}
$$

with $x_a^{i_1\cdots i_k}:=x_a^{i_1}\ldots x_a^{i_k}$ is the source moment tensor.

Definition 3.8 (Scaled derivative tensor). The kth-order scaled derivative tensor in the Cartesian basis is defined as

$$
D'_{i_1\cdots i_k}:=D_{i_1\cdots i_k}\times r^{2k+1}
\tag{3.10}
$$

where the factor r^{2k+1} makes sure D' has non-negative power in r.

Result 3.9. Using the scaled derivative tensor D', the expansion of φ_a can be expressed as

$$
\varphi_a(\vec{x}) = e_a\sum_{k=0}^{\infty}\frac{1}{r^{k+1}k!}n_a^{i_1\cdots i_k}(\vec{x})D'_{i_1\cdots i_k},
\tag{3.11}
$$

where $n_a:=x_a/r^k$ is the *normalized* source moment tensor.

Motivation 3.10. We have worked out an expansion, but can we see from Equation (3.11) that the low-order terms really dominate the series? Physically this makes sense: seen from very far apart, any charge distribution becomes a point charge. Let us verify this intuition.

The object $n_a^{i_1\cdots i_k}$ requires no attention, since it is already normalized by r, which we assume to be much larger than $r_a:=|\vec{x}_a|$. With increasing k it will only become smaller.

The tensor D' has a leading term with no r dependence:

$$
(2k-1)!!\times x^{i_1}\ldots x^{i_k}.
\tag{3.12}
$$

This term is manifestly not traceless: tracing over any pair of indices will produce a prefactor r^2. The other terms in D', therefore, are responsible for subtracting away such traces. For $k=2m$, the highest-power trace term is produced by taking m traces over m pairs of indices, and is of power $2m=k$. Therefore,

$$
D' = \text{leading symmetric term} - (\text{trace terms})
\tag{3.13}
$$

where the leading symmetric term scales as r^k and any trace term scaling as no more than r^k. This warrants no mathematical proof, but this mental model helps us understand that the scaling of D' is contained by r^k.

Result 3.11. $D' \sim r^k$ is suppressed by the prefactor $1/r^{k+1}$, and since the tensor n_a decreases with larger k, Equation (3.11) is indeed dominated by low-order terms.

3.2. Multipole Moment Tensor

Motivation 3.12. In Equation (3.11), the dependence of φ on $\vec{x}$ is not very clear: at order k, this dependence is proportional to

$$n_a^{i_1 \cdots i_k} D'_{i_1 \cdots i_k} \tag{3.14}$$

where D' is rather involved. We want an expression of φ whose dependence on $\vec{x}$ is clear, and upon fixing $\vec{x}$, would give a physical quantity that depends on $\vec{x}_a$.

More specifically, we want to move the complexity of D' to n_a. A tempting approach is to replace the tensor

$$n_a^{i_1 \cdots i_k} := \frac{x_a^{i_1 \cdots i_k}}{r^k} \tag{3.15}$$

with the actual normalized position tensor

$$n^{i_1 \cdots i_k} := \frac{x^{i_1 \cdots i_k}}{r^k}, \tag{3.16}$$

and at the same time, replace D' with a tensor that depends on $\vec{x}_a$.

Notation 3.13. The vector $\vec{y}$ labels the variable of D' *before taking the derivative,* note that it will always be set to $\vec{x}$ after taking the derivative. It is interchangeable with $\vec{x}$ in the following.

From Equation (3.12), The tensor $D'_{i_1, \cdots, i_k}$ has leading term proportional to $x^{i_1 \cdots i_k}$, so for this term we can indeed swap

$$x_a^{i_1 \cdots i_k} \leftrightarrow x^{i_1 \cdots i_k} \tag{3.17}$$

to obtain

$$(2k-1)!! \times \frac{x^{i_1 \cdots i_k}}{r^k} x_a^{i_1 \cdots i_k} =: (2k-1)!! \times n^{i_1 \cdots i_k} x_a^{i_1 \cdots i_k} \tag{3.18}$$

which is the leading term of Equation (3.14). Its sub-leading terms are more complicated. The m-th sub-leading term is the one with m Kronecker deltas:

$$(-1)^k (-1)^{k-m} (2k-2m-1)!! \times r^{2m} \delta_{i_1, j_1} \cdots \delta_{i_m, j_m} \times y^{\ell_1} \cdots y^{\ell_{k-2m}}, \tag{3.19}$$

where the leading $(-1)^k$ comes from the definition of D'. Each $\delta_{i,j}$ is due to ∂_j acting on the denominator, followed by ∂_i that acts on the numerator, so each δ means *one less deriv-*

ative operator acting on the denominator. This manifests in the coefficient $(-1)^{k-m}(2k-2m-1)!!$. The indices $(\ell_1, \cdots \ell_{k-2m})$ are the ones that survived the m contractions.

Recall that we want to extract $y^{i_1 \cdots i_k}$ from this term in D'. We assign the delta symbols and prefactors in Equation (3.19) to n_a and focus on the remaining part

$$r^{2m} y^{\ell_1} \cdots y^{\ell_{k-2m}} \tag{3.20}$$

Our task of constructing $x^{i_1, \cdots, i_k}$ from this term is faced with a difficulty: Equation (3.19) has $k - 2m < k$ indices on y. We overcome this by manually restoring $2m$ indices, using

$$y^{\ell_1, \cdots, \ell_{k-2m}} r^{2m} = y^{\ell_1, \cdots, \ell_k} \times \delta_{i_1, j_1} \times \cdots \times \delta_{i_m, j_m}. \tag{3.21}$$

Now, these y components can be combined with $1/r^k$ to form n.

From the perspective of n_a, it received the prefactors and δ symbols, which would cause it to contract m pairs and produce r_a^{2m} with $r_a := |\vec{x}_a|$. Then it received some δ symbols again, this time generated from the index-restore trick Equation (3.21). Finally, it donated its $1/r^k$ so that the extracted k-th order y tensor becomes n.

In all, at order k, for its product with the mth term, the n_a tensor has become

$$(-1)^k (-1)^{k-m} (2k-2m-1)!! \times r_a^{2m} \delta_{i_1, j_1} \cdots \delta_{i_m, j_m} \times x_a^{\ell_1} \cdots x_a^{\ell_{k-2m}}, \tag{3.22}$$

which is *exactly the same* as the general term in D', except for the new subscripts a. The leading term is a "sub-leading" term with $m = 0$. We have completed the task to "move" the complexity from D' to another object: the following swap

$$\begin{gathered} \text{Within the } m\text{th term,} \\ D' \mapsto n, \quad n_a \mapsto \text{Equation (3.22)} \end{gathered} \tag{3.23}$$

leaves $\varphi_a(\vec{x})$ unchanged.

Definition 3.14 (Multipole Tensor). The k-th order multipole tensor Q is defined as

$$Q := \sum_{m=0}^{\lfloor k/2 \rfloor} (-1)^m (2k-2m-1)!! \times r_a^{2m} \delta_{i_1, j_1} \cdots \delta_{i_m, j_m} \times x_a^{\ell_1} \cdots x_a^{\ell_{k-2m}}, \tag{3.24}$$

where the indices $(\ell_1, \cdots, \ell_{k-2m})$ are the ones except those inside δ symbols, and

$$\lfloor k/2 \rfloor \equiv \min\{m \mid m \in \mathbb{Z}, m \geq k/2\} \tag{3.25}$$

is the lower-rounded integer of $k/2$.

Result 3.15. Using the multipole moment tensor Q, the expansion of φ_a can be expressed as

$$\varphi_a(\vec{x}) = e_a \sum_{k=0}^{\infty} \frac{1}{r^{k+1} k!} n^{i_1 \cdots i_k}(\vec{x}) Q_{i_1 \cdots i_k}, \tag{3.26}$$

where $n := x/r^k$ is the *normalized* field (as in "far-field") moment tensor.

Remark 3.16. This expression is physically very clear. At order k, for a given spatial moment component $n^{i_1 \cdots i_k}$, the tensor Q completely specifies the contribution to φ_a by this moment n.

Remark 3.17. Our swap in Equation (3.23) worked because the tensor $y^{i_1 \cdots i_k}$ can be factored out from *every term* in D'. This is due to the specific structure of the derivative of $1/|\vec{y}|$. For a generic potential φ, we can Taylor expand it, but might not be able to write it in a form similar to Result 3.15.

4. Dive Into the Code

This project folder is organized as follows. The main package `multipole_expansion` contains the core implementation, while test scripts and documentation live at the top level.

```
multipole_expansion/
├── .venv/                  # Virtual environment (created by uv)
├── pyproject.toml          # Project configuration
├── multipole_expansion/    # Main package
│   ├── multipole_moments.py      # Q tensor construction
│   ├── taylor_expansion.py       # Taylor series approach
│   ├── derivatives.py            # Derivatives of 1/r
│   ├── contraction.py            # Einstein summation
│   └── ...
├── run_demo.py            # Basic demonstration
├── test_higher_orders.py  # High-order verification
└── ...
```

Although `uv run <file>` is the most convenient way to run python files, when viewing code and using the language server features of your code editor (such as "go to definition"), it is helpful to make your editor aware of the locally specified environment.

For this purpose, you can run `source .venv/bin/activate`. After that, the command-line prompt should look like `(multipole-expansion) → multipole_expansion`, and "go to definition" will help you jump between files.

4.1. The Main Script

The python module `multipole_expansion` is written inside the folder with the same name. The key python file is `multipole_expansion/multipole_moments.py`, while other python files provide utility functions, such as contraction for Einstein summation and derivatives for computing the derivatives of $1/r$.

Let us start from `multipole_moments.py` and explore the code base.

Go to line 138. The function `_Q_n_general` returns our desired symmetric traceless tensor. What it does is construct the main product term with coefficient $(2n-1)!!$ and then systematically subtract away the trace correction terms. Let us examine how these terms are constructed.

Go to line 169. The function `_compute_all_traces` computes all the trace correction terms that must be subtracted to make the tensor traceless. For each order n, we can have up to $\lfloor n/2 \rfloor$ pairs of contracted indices, and this function systematically generates all such contractions by summing over

$$k \equiv \text{number of contracted index pairs.} \tag{4.1}$$

Each pairing is implemented as an array with length k,

```
pairing = [ (i_1,j_1) , ... , (i_k,j_k) ]
```

where each element is a tuple, and $\forall \ell \in [1, k], i_\ell < j_\ell$.

Denoting the set of pairings which involve k pairs as $\mathbb{P}_k$, we expand

$$\begin{aligned} \text{All traces} &= \sum_k \quad \text{traces with } k \text{ pairs} \\ &= \sum_k \sum_{\text{pairing} \in \mathbb{P}_k} \text{trace-term}(\text{ pairing }), \end{aligned} \tag{4.2}$$

where the function

$$\begin{aligned} \text{trace-term}(\bullet) : \text{pairing} \mapsto (-1)^k (2n-2k-1)!! \\ \times r^{2k} \delta_{i_1,j_1} \ldots \delta_{i_k,j_k} x^{\ell_1} \ldots x^{\ell_{n-2k}} \end{aligned} \tag{4.3}$$

is implemented after line 189. The function `_trace_with_k_pairs` handles the construction of trace terms with exactly k pairs of Kronecker deltas. For example, when $k = 1$, we get terms like

$$\delta^{ij} x_a^k x_a^l \tag{4.4}$$

multiplied by $|x_a|^2$ and the coefficient $(2n-3)!!$. When $k = 2$, we get two deltas, a factor of $|x_a|^4$, and coefficient $(2n-5)!!$. The coefficient formula $\text{coeff} = (2n-2k-1)!!$ is what makes the Taylor expansion and Q tensor formulations mathematically equivalent.

Now, the only remaining task is to construct the set of pairings $\mathbb{P}_k$ for arbitrary order n. This is implemented at line 234, function `_generate_k_pairings`, which is a *recursive algorithm* that

- picks *one* pair of indices from a list of indices,
- shortens the list and updates $k \mapsto k-1$,
- iterates (recursively runs) on the shortened list,
- maps the new indices to the old ones (those in the initial list of indices).

Let us illustrate the last step. For example, if $k = 2$ and we choose among 4 indices, then after the first iteration which picks `[1,2]` out, there will remain 2 indices,

```
remaining = [3,4]; remaining[1] = 3, remaining[2] = 4
```

and the iteration will output a pair `(1,2)` which actually means

```
(remaining[1],remaining[2]) = [3,4]
```

so we need to "map new indices to old ones". Once the condition

```
len(remaining) ⩾ 2 * (k - 1) # See `multipole_moments` line 264
```

is violated, the iteration will stop.

4.2. The Derivative Approach

In addition to the Q tensor construction, we also implement the multipole expansion using the Taylor series derivative approach. Go to `multipole_expansion/derivatives.py`, line 145. The function `_build_derivative_numerator` constructs the polynomial that appears in the numerator when we take the n-th derivative of $1/r$. The formula is

$$\partial^n(1/r) = \left((-1)^n/r^{2n+1}\right) \times P^{i_1 \cdots i_n} \tag{4.5}$$

where P is a polynomial with the same coefficient structure as the Q tensor: $(2n-1)!!$ for the main term, $(2n-3)!!$ for single-pair traces, and so on.

This parallel structure between the derivative and the Q tensor is not a coincidence. It reflects the fundamental mathematical relationship between the two formulations of multipole expansion. When we compute $\phi^{(n)} = ((-1)^n/n!)x_a^{i_1 \cdots i_n}\partial^n(1/r)$ using the Taylor approach, the two $(-1)^n$ factors cancel, leaving us with $(1/n!)$ times a polynomial with the same coefficient structure as Q. After index contraction, both approaches yield identical results.

4.3. Symbolica Basics

Up to now, everything seems perfectly mathematical; we are just telling Python to carry out some mathematical operations. The "symbolic" infrastructure that implements the Kronecker deltas and $\vec{x}, \vec{n}$ vectors are just written as `self.delta`, `self.xa`, etc. But what exactly are these? How did we construct them?

Let us use the Einstein notation module `multipole_expansion/contraction.py` as an example. Go to line 128 of `contraction.py`. The function `_contract_delta` defines the behavior of the Kronecker delta. Because our calculations only involve the position vector x and the unit vector n, the Kronecker delta will be completely defined as long as it can contract with these two vectors. Besides, we also specify that the contraction of a Kronecker delta with itself is $\delta^{ii} = 1 \times 3 = 3$, which is the 3D trace.

In a similar manner, we define in this file other contraction rules, such as those of x with itself or with n. We then wrap all of these contractions into the function at line 55, `contract_indices`, which returns a fully contracted expression. The contraction engine applies pattern matching repeatedly until no more contractions are possible, which is why we sometimes need multiple passes for high-order multipole moments.

In `contraction.py`, you may have noticed the "wildcard" pattern in `symbolica` is similar to that in Wolfram Mathematica: `x_` is a pattern wildcard that matches any "single-atom" expression in the tree-representation. This pattern matching capability is what allows us to define Einstein summation rules declaratively rather than imperatively.

We have seen that symbolic computation boils down to a set of pattern matching and replacing rules, and that with a recursive algorithm, we can construct arbitrarily high-ordered terms from lowest-order ones. The derivative polynomial is built recursively by including all possible delta contractions. This recursive structure extends to arbitrarily high orders, limited only by computational resources rather than conceptual complexity.

5. Troubleshooting

Symbolica is not a small package. Although it has pre-built binaries for the platforms it supports, download and install can take a minute or two. If Symbolica download upon running `uv sync` is slow, you can try installing with pip directly after activating the virtual environment.

```
source .venv/bin/activate
pip install --no-cache-dir symbolica
```

To verify the `symbolica` install, activate the virtual environment and test the import. If everything is working correctly, you should see a confirmation message.

```
source .venv/bin/activate
python -c "from symbolica import Expression; print('Symbolica OK')"
```

You can also run the full demo to see the multipole expansion in action, or run the higher-order tests to verify that the implementation handles arbitrary orders correctly.

```
python run_demo.py
python test_higher_orders.py
```